Лабораторна робота № 1.

Тема: Засоби створення багатопоточних програм.

Мета: Познайомитись із засобами побудови паралельних програм різними мовами програмування.

Теоретичні відомості.

Переважна більшість сучасних версій більшості популярних мов програмування мають засоби для розробки багатопоточних (паралельних) програм. Ці засоби або відразу вбудовані в мову програмування (Java, Ada), або реалізовані за допомогою розширення бібліотеки оточення додатковими класами та/або методами (C++, C#, Pascal та ін).

Так, у мові Java базовим для розпаралелювання є клас Thread, нащадки якого можуть виконуватись у окремих потоках. Аналогічні класи були також додані у мови C++, C#, Pascal. А от у мові програмування Ada для реалізації паралелізму використовуються окремі програмні структури – задачі (task). Класи, схожі за способом організації розпаралелювання із задачами були додані у мову C#. Також у мови програмування C++, C#, Pascal були додані класи, які реалізують семафори, монітори і мютекси та інші засоби та примітиви, що має на меті полегшити розробку паралельних програм цими мовами.

Завдання до лабораторної роботи.

1. Ознайомтесь із можливостями класів Thread у мовах програмування Java (https://www.bestprog.net/uk/2021/01/17/java-java-language-tools-for-working-with-threads-ua/ або https://www.guru99.com/uk/multithreading-java.html) та C# (https://abitap.com/21-1- vvedennya-u-bagatopotochnist-klas-thread/), а також задач у мові Ada (http://boris.cdu.edu.ua/ada/part_1/ch_15.html).

2. Орієнтуючись на приклади (https://github.com/oboris/threaddemo – Java, https://github.com/oboris/threaddemosharp – C#, https://github.com/oboris/threaddemoada – Ada), створіть прогами на мовах C#, Java та Ada, які б у різних потоках (кількість потоків можна змінювати) знаходили суми числових послідовностей, які починаються з нуля і зростають із заданим кроком. Завершення роботи потоків має здійснюватися у окремому, керуючому потоці (див. приклади) шляхом генерування дозволу через заданий проміжок часу для кожного потоку окремо. По завершенню роботи кожен потік має вивести на екран свій порядковий номер, знайдену суму та кількість використаних елементів послідовності (доданків).

3. Запустіть створені програми з різною кількістю потоків, у яких реалізовані знаходження сум числових послідовностей.

4. Підготуйте звіт з лабораторної роботи у якому мають бути скріншоти результатів роботи програм з різною кількістю потоків та скріншоти диспетчера задач з демонстрацією завантаженостей ядер процесора для кожного варіанту паралельної програми.

5. Вихідні коди текстів програм розмістіть на Git-репозиторії та вкажіть відповідні посилання у звіті.

Код виконаних проэктів:

Посилання на Git:

https://github.com/IvanTukalo/POP1/tree/master/Lab1

C#:

```
using System;
using System.Threading;
using System.Numerics;

namespace Lab1Csharp
{
    class Program
    {
        static void Main(string[] args)
        {
            Thread.CurrentThread.CurrentCulture = new System.Globalization.CultureInfo("en-US");
            Console.OutputEncoding = System.Text.Encoding.UTF8;
            Console.ForegroundColor = ConsoleColor.Black;
            Console.BackgroundColor = ConsoleColor.White;
            Console.Clear();

            while (true)
            {
                int step;
                while (true)
                {
                    Console.WriteLine("Введіть крок роботи потоків");
                    if (int.TryParse(Console.ReadLine(), out step) && step > 0)
                    {
                        break;
                    }
                    Console.WriteLine("Помилка вводу. Будь ласка, введіть ціле додатне число.");
                }

                int[] times;
                while (true)
                {
                    Console.WriteLine("Введіть час роботи потоків у секундах через пробіл");
                    string input = Console.ReadLine();

                    if (string.IsNullOrWhiteSpace(input))
                    {
                        continue;
                    }

                    string[] timesStr = input.Split(new char[] { ' ' },
StringSplitOptions.RemoveEmptyEntries);
                    times = new int[timesStr.Length];
                    bool allValid = true;

                    for (int i = 0; i < timesStr.Length; i++)
                    {
                        if (!int.TryParse(timesStr[i], out times[i]) || times[i] <= 0)
                        {
                            allValid = false;
                            break;
                        }
                    }

                    if (allValid)
                    {
                        break;
                    }
                    Console.WriteLine("Помилка вводу. Переконайтеся, що ви ввели лише цілі додатні
числа через пробіл.");
                }

                int numThreads = times.Length;

                Worker[] workers = new Worker[numThreads];
                Thread[] workerThreads = new Thread[numThreads];
                Thread[] stopperThreads = new Thread[numThreads];
```

```
            for (int i = 0; i < numThreads; i++)
            {
                int timeLimit = times[i];
                int threadId = i + 1;

                workers[i] = new Worker(threadId, step, timeLimit);
                workerThreads[i] = new Thread(workers[i].Calculate);

                int currentIndex = i;
                stopperThreads[i] = new Thread(() => {
                    Thread.Sleep(timeLimit * 1000);
                    workers[currentIndex].Stop();
                });
            }

            for (int i = 0; i < numThreads; i++)
            {
                workerThreads[i].Start();
                stopperThreads[i].Start();
            }

            for (int i = 0; i < numThreads; i++)
            {
                workerThreads[i].Join();
            }

            Console.WriteLine("Усі потоки завершили роботу. Починаємо новий цикл.");
            Console.WriteLine();
        }
    }
}

class Worker
{
    private int id;
    private int step;
    private int time;
    private volatile bool canStop = false;

    public Worker(int id, int step, int time)
    {
        this.id = id;
        this.step = step;
        this.time = time;
    }

    public void Stop()
    {
        canStop = true;
    }

    public void Calculate()
    {
        BigInteger sum = 0;
        BigInteger elementsCount = 0;

        while (!canStop)
        {
            sum += step;
            elementsCount++;
        }

        Console.WriteLine($"{id} - {sum}, {step} - {elementsCount} разів за {time} сек.");
    }
}
    }
```

Java:

```
import java.math.BigInteger;
import java.util.Scanner;
import java.io.PrintStream;
import java.nio.charset.StandardCharsets;

public class Main {
    public static void main(String[] args) {
        System.setOut(new PrintStream(System.out, true, StandardCharsets.UTF_8));
        Scanner scanner = new Scanner(System.in, StandardCharsets.UTF_8);

        while (true) {
            int step = 0;
            while (true) {
                System.out.println("Введіть крок роботи потоків (0 для виходу)");
                String stepInput = scanner.nextLine();
                try {
                    step = Integer.parseInt(stepInput.trim());
                    if (step == 0) {
                        System.out.println("Програма завершує роботу.");
                        return;
                    }
                    if (step > 0) {
                        break;
                    }
                } catch (NumberFormatException e) {

                }
                System.out.println("Помилка вводу. Будь ласка, введіть ціле додатне число або 0
для виходу.");
            }

            int[] times;
            while (true) {
                System.out.println("Введіть час роботи потоків у секундах через пробіл");
                String input = scanner.nextLine();

                if (input == null || input.trim().isEmpty()) {
                    continue;
                }

                String[] timesStr = input.trim().split("\\s+");
                times = new int[timesStr.length];
                boolean allValid = true;

                for (int i = 0; i < timesStr.length; i++) {
                    try {
                        times[i] = Integer.parseInt(timesStr[i]);
                        if (times[i] <= 0) {
                            allValid = false;
                            break;
                        }
                    } catch (NumberFormatException e) {
                        allValid = false;
                        break;
                    }
                }

                if (allValid) {
                    break;
                }
                System.out.println("Помилка вводу. Переконайтеся, що ви ввели лише цілі додатні
числа через пробіл.");
            }

            int numThreads = times.length;
            WorkerThread[] workers = new WorkerThread[numThreads];
            Thread[] stoppers = new Thread[numThreads];

            for (int i = 0; i < numThreads; i++) {
                int timeLimit = times[i];
                workers[i] = new WorkerThread(i + 1, step, timeLimit);
                final WorkerThread worker = workers[i];
```

```
                stoppers[i] = new Thread(() -> {
                    try {
                        Thread.sleep(timeLimit * 1000L);
                    } catch (InterruptedException e) {
                        Thread.currentThread().interrupt();
                    }
                    worker.stopThread();
                });
            }

            for (int i = 0; i < numThreads; i++) {
                workers[i].start();
                stoppers[i].start();
            }

            for (int i = 0; i < numThreads; i++) {
                try {
                    workers[i].join();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }

            System.out.println("Усі потоки завершили роботу. Починаємо новий цикл.");
            System.out.println();
        }
    }
}

class WorkerThread extends Thread {
    private final int id;
    private final int step;
    private final int timeLimit;
    private volatile boolean canStop = false;

    public WorkerThread(int id, int step, int timeLimit) {
        this.id = id;
        this.step = step;
        this.timeLimit = timeLimit;
    }

    public void stopThread() {
        this.canStop = true;
    }

    @Override
    public void run() {
        BigInteger sum = BigInteger.ZERO;
        BigInteger elementsCount = BigInteger.ZERO;
        BigInteger stepBig = BigInteger.valueOf(step);

        while (!canStop) {
            sum = sum.add(stepBig);
            elementsCount = elementsCount.add(BigInteger.ONE);
        }

        System.out.println(id + " - " + sum + ", " + step + " - " + elementsCount + " разів за "
+ timeLimit + " сек.");
    }
}
```

ADA:

```
with Ada.Wide_Text_IO; use Ada.Wide_Text_IO;
with Ada.Strings.Wide_Fixed; use Ada.Strings.Wide_Fixed;
with Ada.Strings; use Ada.Strings;

procedure Lab1Ada is

   protected type Stop_Flag is
      procedure Set;
      function Get return Boolean;
   private
      Value : Boolean := False;
   end Stop_Flag;

   protected body Stop_Flag is
      procedure Set is begin Value := True; end Set;
      function Get return Boolean is begin return Value; end Get;
   end Stop_Flag;

   protected Output_Lock is
      entry Seize;
      procedure Release;
   private
      Locked : Boolean := False;
   end Output_Lock;

   protected body Output_Lock is
      entry Seize when not Locked is
      begin
         Locked := True;
      end Seize;

      procedure Release is
      begin
         Locked := False;
      end Release;
   end Output_Lock;

   type Stop_Flag_Access is access Stop_Flag;

   task type Main_Thread is
      entry Start (Id : Integer; Step : Long_Long_Integer; Time : Integer; Flag :
Stop_Flag_Access);
   end Main_Thread;

   task type Break_Thread is
      entry Start (Seconds : Integer; Flag : Stop_Flag_Access);
   end Break_Thread;

   task body Main_Thread is
      My_Id : Integer;
      My_Step : Long_Long_Integer;
      My_Time : Integer;
```

```
            My_Flag : Stop_Flag_Access;
            Sum : Long_Long_Integer := 0;
            Elements_Count : Long_Long_Integer := 0;
         begin
            accept Start (Id : Integer; Step : Long_Long_Integer; Time : Integer; Flag :
Stop_Flag_Access) do
               My_Id := Id;
               My_Step := Step;
               My_Time := Time;
               My_Flag := Flag;
            end Start;

            loop
               Sum := Sum + My_Step;
               Elements_Count := Elements_Count + 1;
               exit when My_Flag.Get;
            end loop;

            Output_Lock.Seize;
            Put_Line (Trim(Integer'Wide_Image(My_Id), Both) & " - " &
                      Trim(Long_Long_Integer'Wide_Image(Sum), Both) & ", " &
                      Trim(Long_Long_Integer'Wide_Image(My_Step), Both) & " - " &
                      Trim(Long_Long_Integer'Wide_Image(Elements_Count), Both) & " разів за " &
                      Trim(Integer'Wide_Image(My_Time), Both) & " сек.");
            Output_Lock.Release;
         end Main_Thread;

         task body Break_Thread is
            My_Seconds : Integer;
            My_Flag : Stop_Flag_Access;
         begin
            accept Start (Seconds : Integer; Flag : Stop_Flag_Access) do
               My_Seconds := Seconds;
               My_Flag := Flag;
            end Start;

            delay Duration(My_Seconds);
            My_Flag.Set;
         end Break_Thread;

         Step_Input : Long_Long_Integer;
         Input_Line : Wide_String (1 .. 256);
         Last_Idx : Natural;
         Num_Threads : Natural := 0;
         Times : array (1 .. 100) of Integer;

         Valid_Step : Boolean;
         Valid_Times : Boolean;
         I, J : Positive;

      begin
         Main_Loop : loop
            Valid_Step := False;
```

```
            while not Valid_Step loop
               Put_Line ("Введіть крок роботи потоків (0 для виходу)");
               begin
                  Get_Line (Input_Line, Last_Idx);
                  Step_Input := Long_Long_Integer'Wide_Value(Input_Line(1 .. Last_Idx));
                  if Step_Input = 0 then
                     Put_Line ("Програма завершує роботу.");
                     exit Main_Loop;
                  elsif Step_Input > 0 then
                     Valid_Step := True;
                  else
                     Put_Line ("Помилка вводу. Будь ласка, введіть ціле додатне число або 0 для
виходу.");
                  end if;
               exception
                  when others =>
                     Put_Line ("Помилка вводу. Будь ласка, введіть ціле додатне число або 0 для
виходу.");
               end;
            end loop;

            Valid_Times := False;
            while not Valid_Times loop
               Put_Line ("Введіть час роботи потоків у секундах через пробіл");
               Get_Line (Input_Line, Last_Idx);

               Num_Threads := 0;
               I := 1;
               Valid_Times := True;

               if Last_Idx = 0 then
                  Valid_Times := False;
               else
                  while I <= Last_Idx loop
                     while I <= Last_Idx and then Input_Line(I) = ' ' loop
                        I := I + 1;
                     end loop;
                     if I <= Last_Idx then
                        J := I;
                        while J <= Last_Idx and then Input_Line(J) /= ' ' loop
                           J := J + 1;
                        end loop;

                        begin
                           Num_Threads := Num_Threads + 1;
                           Times(Num_Threads) := Integer'Wide_Value(Input_Line(I .. J - 1));
                           if Times(Num_Threads) <= 0 then
                              Valid_Times := False;
                              exit;
                           end if;
                        exception
                           when others =>
                              Valid_Times := False;
```

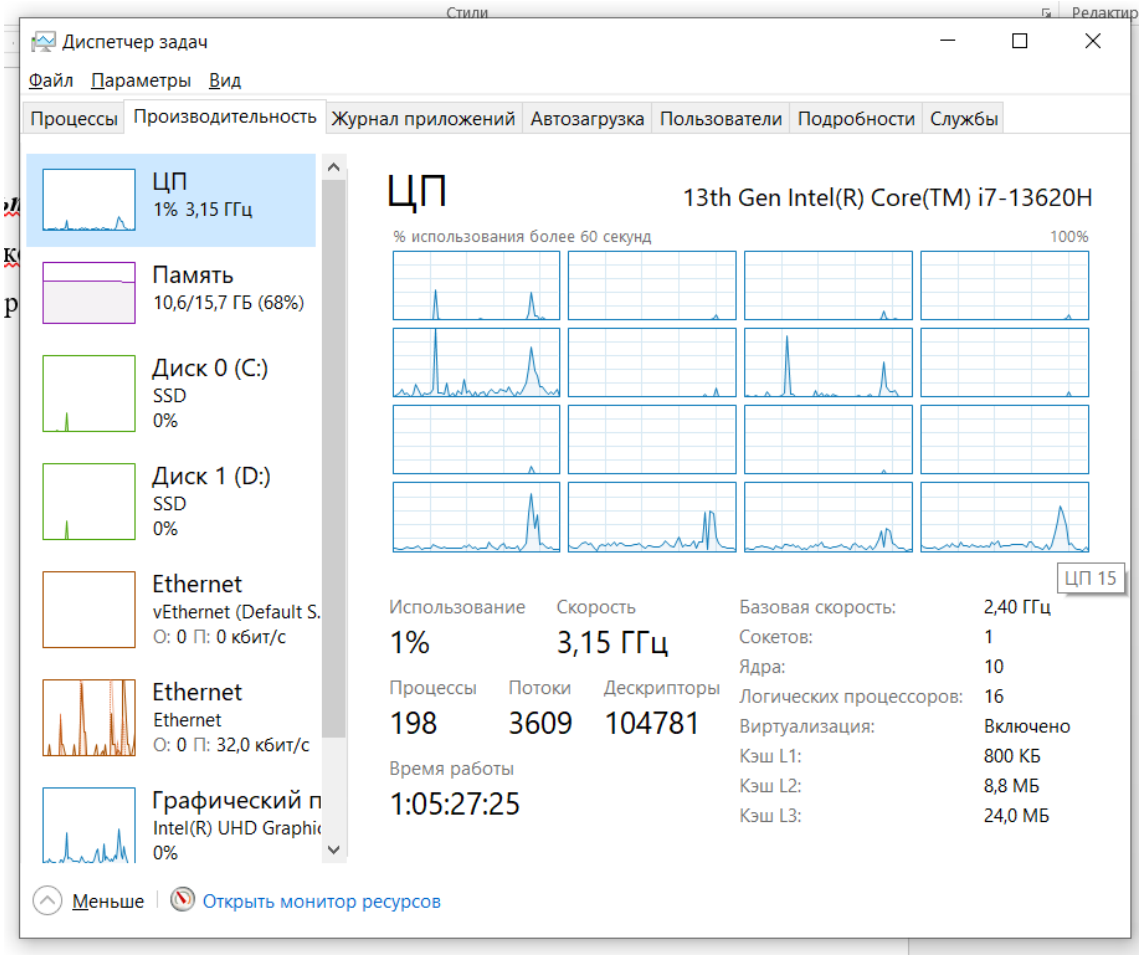
```
                        exit;
                    end;
                    I := J;
                 end if;
              end loop;

              if Num_Threads = 0 then
                 Valid_Times := False;
              end if;
           end if;

           if not Valid_Times then
              Put_Line ("Помилка вводу. Переконайтеся, що ви ввели лише цілі додатні числа
через пробіл.");
           end if;
        end loop;

        declare
           Flags  : array (1 .. Num_Threads) of Stop_Flag_Access;
           Mains  : array (1 .. Num_Threads) of Main_Thread;
           Breaks : array (1 .. Num_Threads) of Break_Thread;
        begin
           for K in 1 .. Num_Threads loop
              Flags(K) := new Stop_Flag;
              Mains(K).Start(K, Step_Input, Times(K), Flags(K));
              Breaks(K).Start(Times(K), Flags(K));
           end loop;
        end;

        Put_Line ("Усі потоки завершили роботу. Починаємо новий цикл.");
        Put_Line ("");
     end loop Main_Loop;
  end Lab1Ada;
```

Результати виконання роботи:

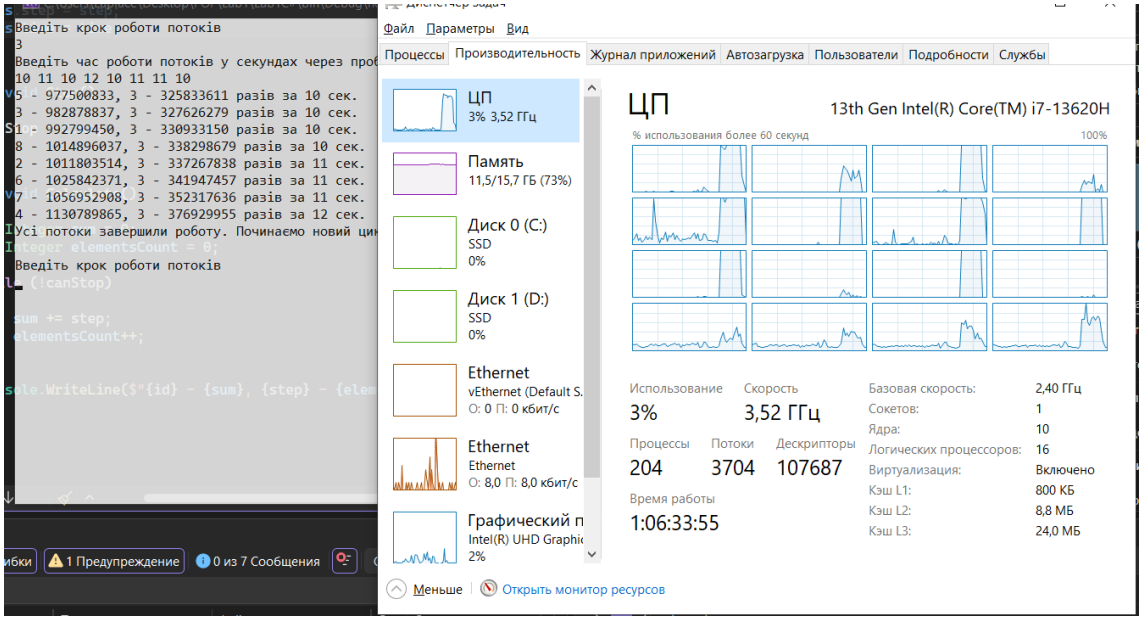
На мойому ноутбуці 10 ядер і 16 логічних процесів. Я порахував доцільним для кожної програми реалізувати 3 сценарія:

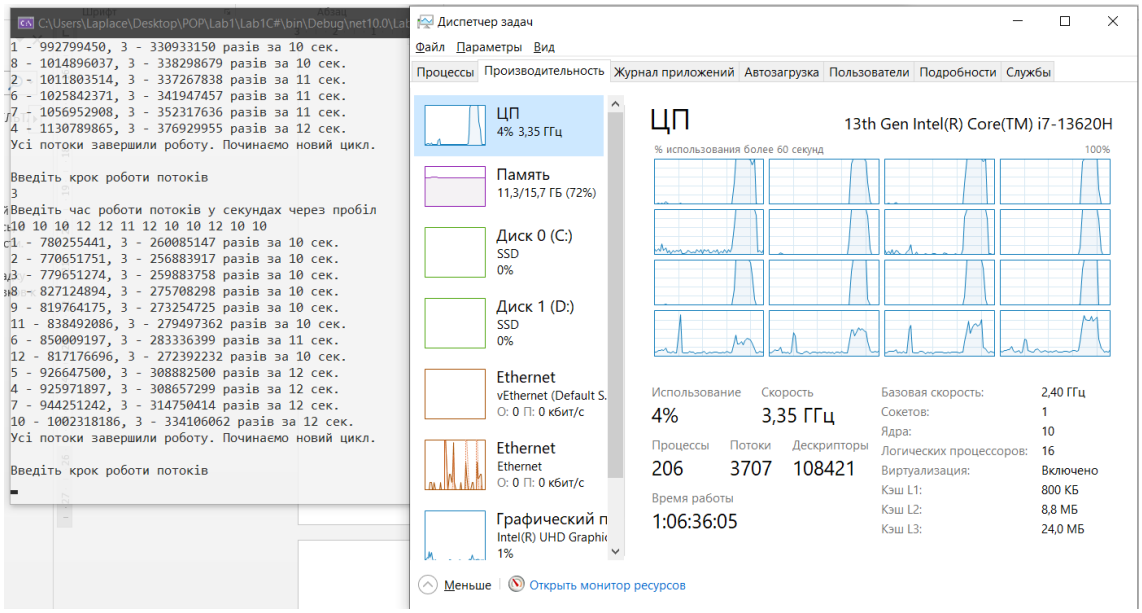
- 8 потоків що менше моєї кількості ядер;
- 12 потоків що більше моєї кількості ядер але менше логічних процесів;
- 18 потоків що більше моєї кількості логічних процесів.

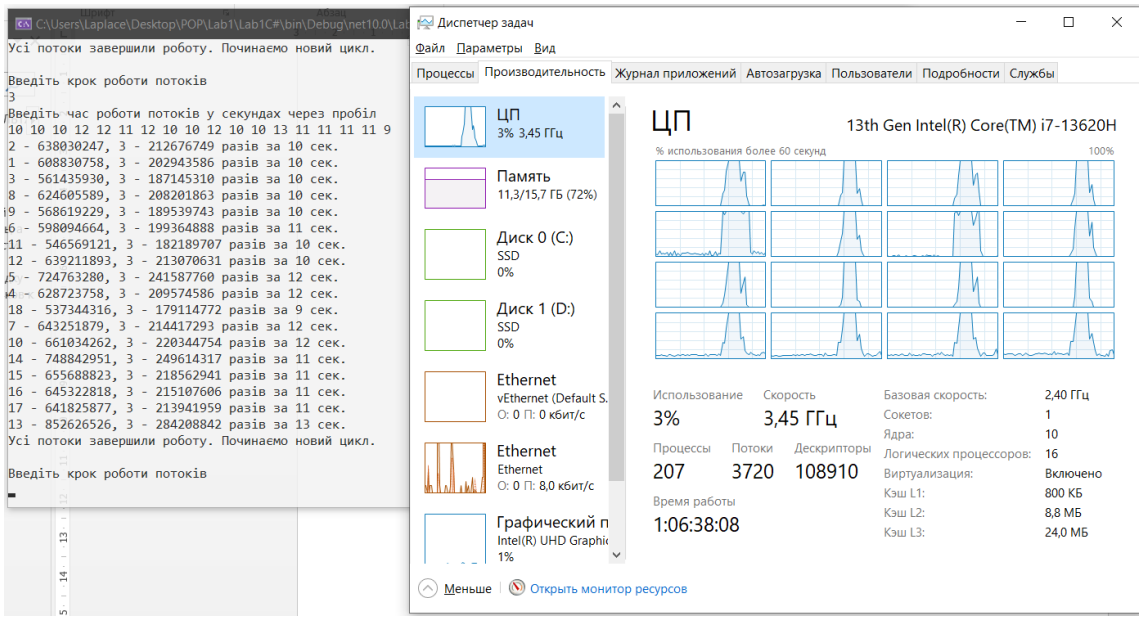
Початковий диспетчер задач:



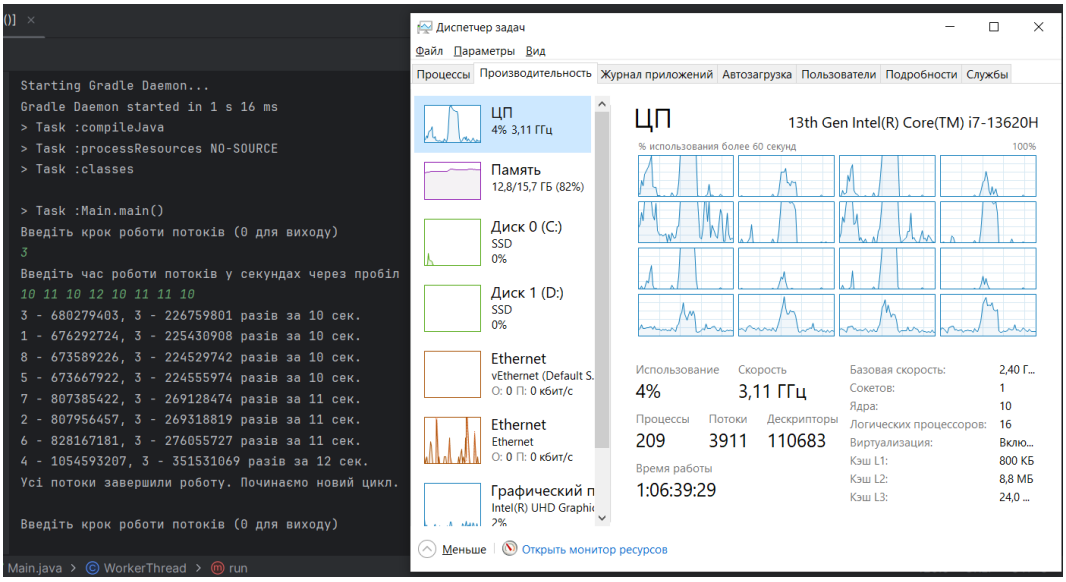
C#:

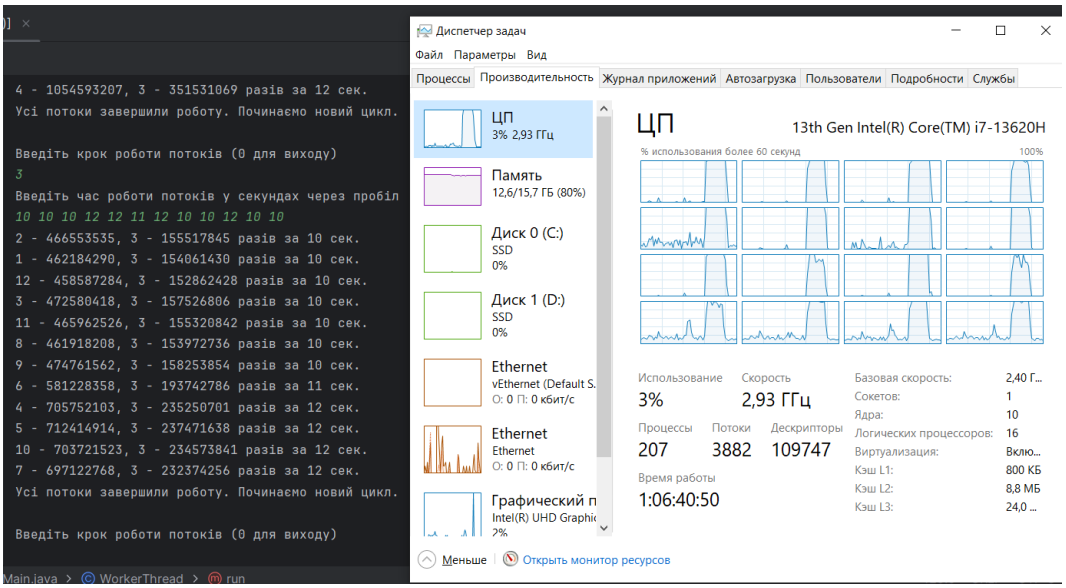

Введіть крок роботи потоків
3
Введіть час роботи потоків у секундах через проб
10 11 10 12 10 11 11 10
5 - 977500833, 3 - 325833611 разів за 10 сек.
3 - 982878837, 3 - 327626279 разів за 10 сек.
1 - 992799450, 3 - 330933150 разів за 10 сек.
8 - 1014896037, 3 - 338298679 разів за 10 сек.
2 - 1011803514, 3 - 337267838 разів за 11 сек.
6 - 1025842371, 3 - 341947457 разів за 11 сек.
7 - 1056952908, 3 - 352317636 разів за 11 сек.
4 - 1130789865, 3 - 376929955 разів за 12 сек.
Усі потоки завершили роботу. Починаємо новий ци
Введіть крок роботи потоків
1 Предупреждение
0 из 7 Сообщения
Файл Параметры Вид
Процессы Производительность Журнал приложений Автозагрузка Пользователи Подробности Службы
ЦП
3% 3,52 ГГц
Память
11,5/15,7 ГБ (73%)
Диск 0 (C:)
SSD
0%
Диск 1 (D:)
SSD
0%
Ethernet
vEthernet (Default S.
О: 0 П: 0 кбит/с
Ethernet
Ethernet
О: 8,0 П: 8,0 кбит/с
Графический п
Intel(R) UHD Graphic
2%
ЦП
13th Gen Intel(R) Core(TM) i7-13620H
% использования более 60 секунд
100%
Использование
3%
Скорость
3,52 ГГц
Процессы
204
Потоки
3704
Дескрипторы
107687
Время работы
1:06:33:55
Базовая скорость: 2,40 ГГц
Сокетов: 1
Ядра: 10
Логических процессоров: 16
Виртуализация: Включено
Кэш L1: 800 КБ
Кэш L2: 8,8 МБ
Кэш L3: 24,0 МБ
Меньше
Открыть монитор ресурсов

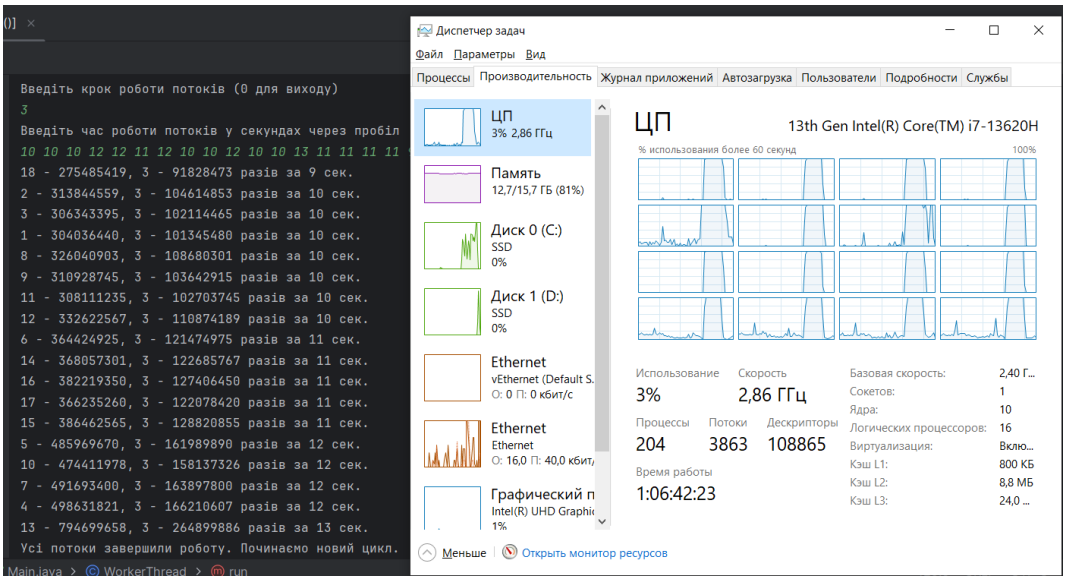

1 - 992799450, 3 - 330933150 разів за 10 сек.
8 - 1014896037, 3 - 338298679 разів за 10 сек.
2 - 1011803514, 3 - 337267838 разів за 11 сек.
6 - 1025842371, 3 - 341947457 разів за 11 сек.
7 - 1056952908, 3 - 352317636 разів за 11 сек.
4 - 1130789865, 3 - 376929955 разів за 12 сек.
Усі потоки завершили роботу. Починаємо новий цикл.
Введіть крок роботи потоків
3
Введіть час роботи потоків у секундах через пробіл
10 10 10 12 12 11 12 10 10 12 10 10
1 - 780255441, 3 - 260085147 разів за 10 сек.
2 - 770651751, 3 - 256883917 разів за 10 сек.
3 - 779651274, 3 - 259883758 разів за 10 сек.
8 - 827124894, 3 - 275708298 разів за 10 сек.
9 - 819764175, 3 - 273254725 разів за 10 сек.
11 - 838492086, 3 - 279497362 разів за 10 сек.
6 - 850009197, 3 - 283336399 разів за 11 сек.
12 - 817176696, 3 - 272392232 разів за 10 сек.
5 - 926647500, 3 - 308882500 разів за 12 сек.
4 - 925971897, 3 - 308657299 разів за 12 сек.
7 - 944251242, 3 - 314750414 разів за 12 сек.
10 - 1002318186, 3 - 334106062 разів за 12 сек.
Усі потоки завершили роботу. Починаємо новий цикл.
Введіть крок роботи потоків
Диспетчер задач
Файл Параметры Вид
Процессы Производительность Журнал приложений Автозагрузка Пользователи Подробности Службы
ЦП
4% 3,35 ГГц
Память
11,3/15,7 ГБ (72%)
Диск 0 (C:)
SSD
0%
Диск 1 (D:)
SSD
0%
Ethernet
vEthernet (Default S.
О: 0 П: 0 кбит/с
Ethernet
Ethernet
О: 0 П: 0 кбит/с
Графический п
Intel(R) UHD Graphic
1%
ЦП
13th Gen Intel(R) Core(TM) i7-13620H
% использования более 60 секунд
100%
Использование
4%
Скорость
3,35 ГГц
Процессы
206
Потоки
3707
Дескрипторы
108421
Время работы
1:06:36:05
Базовая скорость: 2,40 ГГц
Сокетов: 1
Ядра: 10
Логических процессоров: 16
Виртуализация: Включено
Кэш L1: 800 КБ
Кэш L2: 8,8 МБ
Кэш L3: 24,0 МБ
Меньше
Открыть монитор ресурсов


Усі потоки завершили роботу. Починаємо новий цикл.
Введіть крок роботи потоків
3
Введіть час роботи потоків у секундах через пробіл
10 10 10 12 12 11 12 10 10 12 10 10 13 11 11 11 11 9
2 - 638030247, 3 - 212676749 разів за 10 сек.
1 - 608830758, 3 - 202943586 разів за 10 сек.
3 - 561435930, 3 - 187145310 разів за 10 сек.
8 - 624605589, 3 - 208201863 разів за 10 сек.
9 - 568619229, 3 - 189539743 разів за 10 сек.
6 - 598094664, 3 - 199364888 разів за 11 сек.
11 - 546569121, 3 - 182189707 разів за 10 сек.
12 - 639211893, 3 - 213070631 разів за 10 сек.
5 - 724763280, 3 - 241587760 разів за 12 сек.
4 - 628723758, 3 - 209574586 разів за 12 сек.
18 - 537344316, 3 - 179114772 разів за 9 сек.
7 - 643251879, 3 - 214417293 разів за 12 сек.
10 - 661034262, 3 - 220344754 разів за 12 сек.
14 - 748842951, 3 - 249614317 разів за 11 сек.
15 - 655688823, 3 - 218562941 разів за 11 сек.
16 - 645322818, 3 - 215107606 разів за 11 сек.
17 - 641825877, 3 - 213941959 разів за 11 сек.
13 - 852626526, 3 - 284208842 разів за 13 сек.
Усі потоки завершили роботу. Починаємо новий цикл.
Введіть крок роботи потоків
Диспетчер задач
Файл Параметры Вид
Процессы Производительность Журнал приложений Автозагрузка Пользователи Подробности Службы
ЦП
3% 3,45 ГГц
Память
11,3/15,7 ГБ (72%)
Диск 0 (C:)
SSD
0%
Диск 1 (D:)
SSD
0%
Ethernet
vEthernet (Default S.
О: 0 П: 0 кбит/с
Ethernet
Ethernet
О: 0 П: 8,0 кбит/с
Графический п
Intel(R) UHD Graphic
1%
ЦП
13th Gen Intel(R) Core(TM) i7-13620H
% использования более 60 секунд
100%
Использование
3%
Скорость
3,45 ГГц
Процессы
207
Потоки
3720
Дескрипторы
108910
Время работы
1:06:38:08
Базовая скорость: 2,40 ГГц
Сокетов: 1
Ядра: 10
Логических процессоров: 16
Виртуализация: Включено
Кэш L1: 800 КБ
Кэш L2: 8,8 МБ
Кэш L3: 24,0 МБ
Меньше
Открыть монитор ресурсов

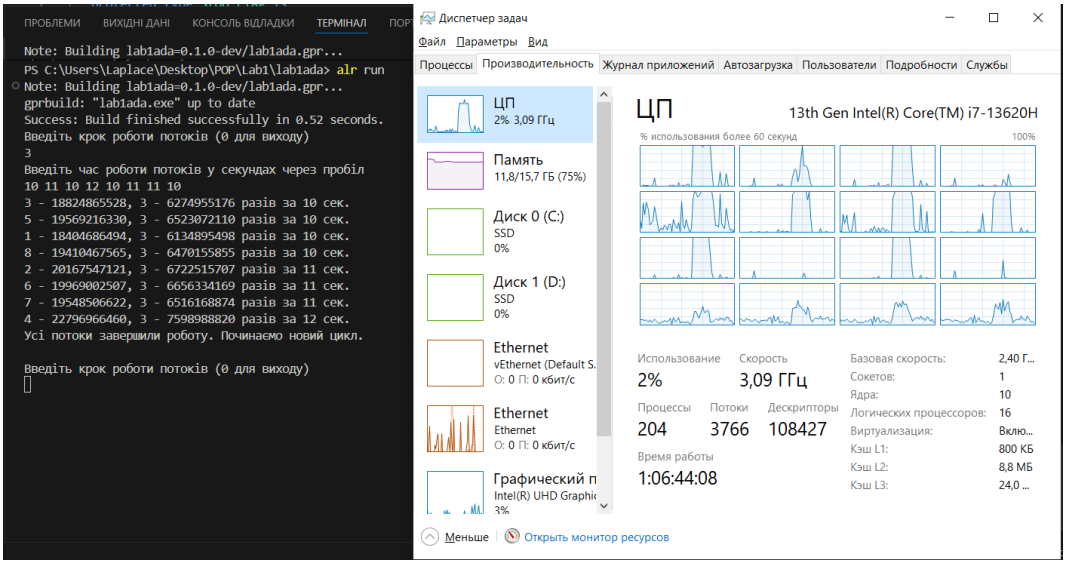
Java:

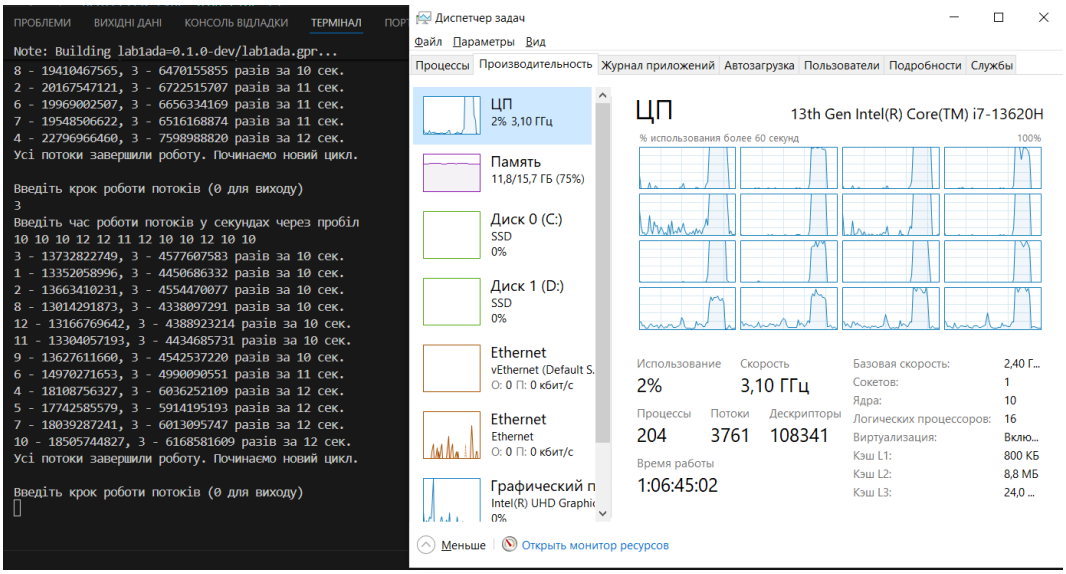

Starting Gradle Daemon...
Gradle Daemon started in 1 s 16 ms
> Task :compileJava
> Task :processResources NO-SOURCE
> Task :classes
> Task :Main.main()
Введіть крок роботи потоків (0 для виходу)
3
Введіть час роботи потоків у секундах через пробіл
10 11 10 12 10 11 11 10
3 - 680279403, 3 - 226759801 разів за 10 сек.
1 - 676292724, 3 - 225430908 разів за 10 сек.
8 - 673589226, 3 - 224529742 разів за 10 сек.
5 - 673667922, 3 - 224555974 разів за 10 сек.
7 - 807385422, 3 - 269128474 разів за 11 сек.
2 - 807956457, 3 - 269318819 разів за 11 сек.
6 - 828167181, 3 - 276055727 разів за 11 сек.
4 - 1054593207, 3 - 351531069 разів за 12 сек.
Усі потоки завершили роботу. Починаємо новий цикл.
Введіть крок роботи потоків (0 для виходу)
Main.java > WorkerThread > run
Диспетчер задач
Файл Параметры Вид
Процессы Производительность Журнал приложений Автозагрузка Пользователи Подробности Службы
ЦП 4% 3,11 ГГц
Память 12,8/15,7 ГБ (82%)
Диск 0 (C:) SSD 0%
Диск 1 (D:) SSD 0%
Ethernet vEthernet (Default S. О: 0 П: 0 кбит/с
Ethernet Ethernet О: 0 П: 0 кбит/с
Графический п Intel(R) UHD Graphic 2%
ЦП 13th Gen Intel(R) Core(TM) i7-13620H
% использования более 60 секунд 100%
Использование 4%
Скорость 3,11 ГГц
Процессы 209
Потоки 3911
Дескрипторы 110683
Время работы 1:06:39:29
Базовая скорость: 2,40 Г...
Сокетов: 1
Ядра: 10
Логических процессоров: 16
Виртуализация: Вклю...
Кэш L1: 800 КБ
Кэш L2: 8,8 МБ
Кэш L3: 24,0 ...
Меньше
Открыть монитор ресурсов

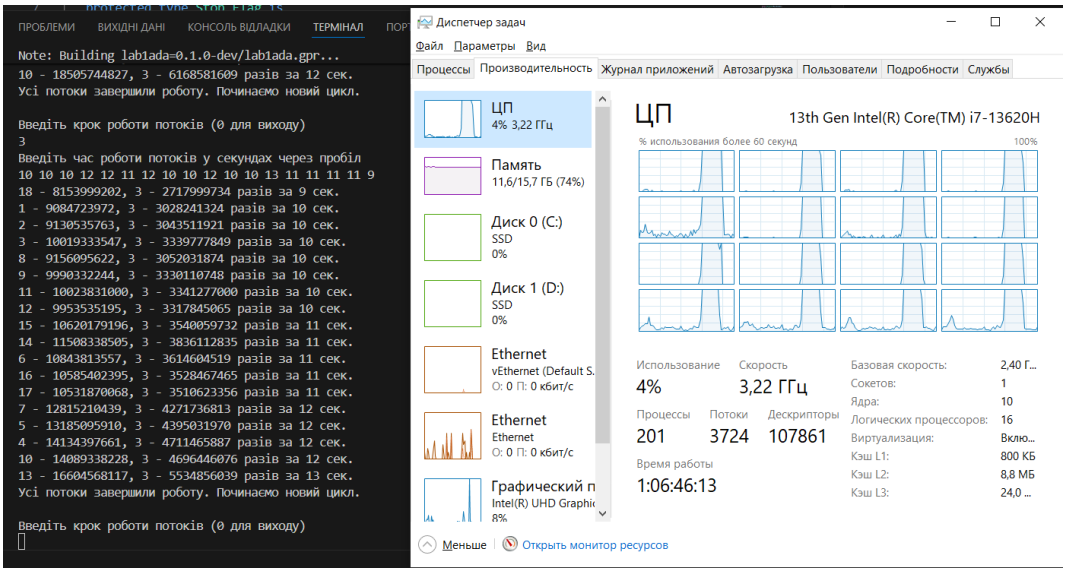

4 - 1054593207, 3 - 351531069 разів за 12 сек.
Усі потоки завершили роботу. Починаємо новий цикл.
Введіть крок роботи потоків (0 для виходу)
3
Введіть час роботи потоків у секундах через пробіл
10 10 10 12 12 11 12 10 10 12 10 10
2 - 466553535, 3 - 155517845 разів за 10 сек.
1 - 462184290, 3 - 154061430 разів за 10 сек.
12 - 458587284, 3 - 152862428 разів за 10 сек.
3 - 472580418, 3 - 157526806 разів за 10 сек.
11 - 465962526, 3 - 155320842 разів за 10 сек.
8 - 461918208, 3 - 153972736 разів за 10 сек.
9 - 474761562, 3 - 158253854 разів за 10 сек.
6 - 581228358, 3 - 193742786 разів за 11 сек.
4 - 705752103, 3 - 235250701 разів за 12 сек.
5 - 712414914, 3 - 237471638 разів за 12 сек.
10 - 703721523, 3 - 234573841 разів за 12 сек.
7 - 697122768, 3 - 232374256 разів за 12 сек.
Усі потоки завершили роботу. Починаємо новий цикл.
Введіть крок роботи потоків (0 для виходу)
Main.java > WorkerThread > run
Диспетчер задач
Файл Параметры Вид
Процессы Производительность Журнал приложений Автозагрузка Пользователи Подробности Службы
ЦП 3% 2,93 ГГц
Память 12,6/15,7 ГБ (80%)
Диск 0 (C:) SSD 0%
Диск 1 (D:) SSD 0%
Ethernet vEthernet (Default S. О: 0 П: 0 кбит/с
Ethernet Ethernet О: 0 П: 0 кбит/с
Графический п Intel(R) UHD Graphic 2%
ЦП 13th Gen Intel(R) Core(TM) i7-13620H
% использования более 60 секунд 100%
Использование 3%
Скорость 2,93 ГГц
Процессы 207
Потоки 3882
Дескрипторы 109747
Время работы 1:06:40:50
Базовая скорость: 2,40 Г...
Сокетов: 1
Ядра: 10
Логических процессоров: 16
Виртуализация: Вклю...
Кэш L1: 800 КБ
Кэш L2: 8,8 МБ
Кэш L3: 24,0 ...
Меньше
Открыть монитор ресурсов


Введіть крок роботи потоків (0 для виходу)
3
Введіть час роботи потоків у секундах через пробіл
10 10 10 12 12 11 12 10 10 12 10 10 13 11 11 11 11
18 - 275485419, 3 - 91828473 разів за 9 сек.
2 - 313844559, 3 - 104614853 разів за 10 сек.
3 - 306343395, 3 - 102114465 разів за 10 сек.
1 - 304036440, 3 - 101345480 разів за 10 сек.
8 - 326040903, 3 - 108680301 разів за 10 сек.
9 - 310928745, 3 - 103642915 разів за 10 сек.
11 - 308111235, 3 - 102703745 разів за 10 сек.
12 - 332622567, 3 - 110874189 разів за 10 сек.
6 - 364424925, 3 - 121474975 разів за 11 сек.
14 - 368057301, 3 - 122685767 разів за 11 сек.
16 - 382219350, 3 - 127406450 разів за 11 сек.
17 - 366235260, 3 - 122078420 разів за 11 сек.
15 - 386462565, 3 - 128820855 разів за 11 сек.
5 - 485969670, 3 - 161989890 разів за 12 сек.
10 - 474411978, 3 - 158137326 разів за 12 сек.
7 - 491693400, 3 - 163897800 разів за 12 сек.
4 - 498631821, 3 - 166210607 разів за 12 сек.
13 - 794699658, 3 - 264899886 разів за 13 сек.
Усі потоки завершили роботу. Починаємо новий цикл.
Main.java > WorkerThread > run
Диспетчер задач
Файл Параметры Вид
Процессы Производительность Журнал приложений Автозагрузка Пользователи Подробности Службы
ЦП 3% 2,86 ГГц
Память 12,7/15,7 ГБ (81%)
Диск 0 (C:) SSD 0%
Диск 1 (D:) SSD 0%
Ethernet vEthernet (Default S. О: 0 П: 0 кбит/с
Ethernet Ethernet О: 16,0 П: 40,0 кбит/
Графический п Intel(R) UHD Graphic 1%
ЦП 13th Gen Intel(R) Core(TM) i7-13620H
% использования более 60 секунд 100%
Использование 3%
Скорость 2,86 ГГц
Процессы 204
Потоки 3863
Дескрипторы 108865
Время работы 1:06:42:23
Базовая скорость: 2,40 Г...
Сокетов: 1
Ядра: 10
Логических процессоров: 16
Виртуализация: Вклю...
Кэш L1: 800 КБ
Кэш L2: 8,8 МБ
Кэш L3: 24,0 ...
Меньше
Открыть монитор ресурсов

ADA:


ПРОБЛЕМИ ВИХІДНІ ДАНІ КОНСОЛЬ ВІДЛАДКИ ТЕРМІНАЛ
Note: Building lab1ada=0.1.0-dev/lab1ada.gpr...
PS C:\Users\Laplace\Desktop\POP\Lab1\lab1ada> alr run
Note: Building lab1ada=0.1.0-dev/lab1ada.gpr...
gprbuild: "lab1ada.exe" up to date
Success: Build finished successfully in 0.52 seconds.
Введіть крок роботи потоків (0 для виходу)
3
Введіть час роботи потоків у секундах через пробіл
10 11 10 12 10 11 11 10
3 - 18824865528, 3 - 6274955176 разів за 10 сек.
5 - 19569216330, 3 - 6523072110 разів за 10 сек.
1 - 18404686494, 3 - 6134895498 разів за 10 сек.
8 - 19410467565, 3 - 6470155855 разів за 10 сек.
2 - 20167547121, 3 - 6722515707 разів за 11 сек.
6 - 19969002507, 3 - 6656334169 разів за 11 сек.
7 - 19548506622, 3 - 6516168874 разів за 11 сек.
4 - 22796966460, 3 - 7598988820 разів за 12 сек.
Усі потоки завершили роботу. Починаємо новий цикл.
Введіть крок роботи потоків (0 для виходу)
Диспетчер задач
Файл Параметры Вид
Процессы Производительность Журнал приложений Автозагрузка Пользователи Подробности Службы
ЦП 2% 3,09 ГГц
Память 11,8/15,7 ГБ (75%)
Диск 0 (C:) SSD 0%
Диск 1 (D:) SSD 0%
Ethernet vEthernet (Default S. О: 0 П: 0 кбит/с
Ethernet Ethernet О: 0 П: 0 кбит/с
Графический п Intel(R) UHD Graphic 3%
ЦП 13th Gen Intel(R) Core(TM) i7-13620H
% использования более 60 секунд 100%
Использование 2%
Скорость 3,09 ГГц
Процессы 204
Потоки 3766
Дескрипторы 108427
Время работы 1:06:44:08
Базовая скорость: 2,40 Г...
Сокетов: 1
Ядра: 10
Логических процессоров: 16
Виртуализация: Вклю...
Кэш L1: 800 КБ
Кэш L2: 8,8 МБ
Кэш L3: 24,0 ...
Меньше
Открыть монитор ресурсов


ПРОБЛЕМИ ВИХІДНІ ДАНІ КОНСОЛЬ ВІДЛАДКИ ТЕРМІНАЛ
Note: Building lab1ada=0.1.0-dev/lab1ada.gpr...
8 - 19410467565, 3 - 6470155855 разів за 10 сек.
2 - 20167547121, 3 - 6722515707 разів за 11 сек.
6 - 19969002507, 3 - 6656334169 разів за 11 сек.
7 - 19548506622, 3 - 6516168874 разів за 11 сек.
4 - 22796966460, 3 - 7598988820 разів за 12 сек.
Усі потоки завершили роботу. Починаємо новий цикл.
Введіть крок роботи потоків (0 для виходу)
3
Введіть час роботи потоків у секундах через пробіл
10 10 10 12 12 11 12 10 10 12 10 10
3 - 13732822749, 3 - 4577607583 разів за 10 сек.
1 - 13352058996, 3 - 4450686332 разів за 10 сек.
2 - 13663410231, 3 - 4554470077 разів за 10 сек.
8 - 13014291873, 3 - 4338097291 разів за 10 сек.
12 - 13166769642, 3 - 4388923214 разів за 10 сек.
11 - 13304057193, 3 - 4434685731 разів за 10 сек.
9 - 13627611660, 3 - 4542537220 разів за 10 сек.
6 - 14970271653, 3 - 4990090551 разів за 11 сек.
4 - 18108756327, 3 - 6036252109 разів за 12 сек.
5 - 17742585579, 3 - 5914195193 разів за 12 сек.
7 - 18039287241, 3 - 6013095747 разів за 12 сек.
10 - 18505744827, 3 - 6168581609 разів за 12 сек.
Усі потоки завершили роботу. Починаємо новий цикл.
Введіть крок роботи потоків (0 для виходу)
Диспетчер задач
Файл Параметры Вид
Процессы Производительность Журнал приложений Автозагрузка Пользователи Подробности Службы
ЦП 2% 3,10 ГГц
Память 11,8/15,7 ГБ (75%)
Диск 0 (C:) SSD 0%
Диск 1 (D:) SSD 0%
Ethernet vEthernet (Default S. О: 0 П: 0 кбит/с
Ethernet Ethernet О: 0 П: 0 кбит/с
Графический п Intel(R) UHD Graphic 0%
ЦП 13th Gen Intel(R) Core(TM) i7-13620H
% использования более 60 секунд 100%
Использование 2%
Скорость 3,10 ГГц
Процессы 204
Потоки 3761
Дескрипторы 108341
Время работы 1:06:45:02
Базовая скорость: 2,40 Г...
Сокетов: 1
Ядра: 10
Логических процессоров: 16
Виртуализация: Вклю...
Кэш L1: 800 КБ
Кэш L2: 8,8 МБ
Кэш L3: 24,0 ...
Меньше
Открыть монитор ресурсов


ПРОБЛЕМИ ВИХІДНІ ДАНІ КОНСОЛЬ ВІДЛАДКИ ТЕРМІНАЛ
Note: Building lab1ada=0.1.0-dev/lab1ada.gpr...
10 - 18505744827, 3 - 6168581609 разів за 12 сек.
Усі потоки завершили роботу. Починаємо новий цикл.
Введіть крок роботи потоків (0 для виходу)
3
Введіть час роботи потоків у секундах через пробіл
10 10 10 12 12 11 12 10 10 12 10 10 13 11 11 11 11 9
18 - 8153999202, 3 - 2717999734 разів за 9 сек.
1 - 9084723972, 3 - 3028241324 разів за 10 сек.
2 - 9130535763, 3 - 3043511921 разів за 10 сек.
3 - 10019333547, 3 - 3339777849 разів за 10 сек.
8 - 9156095622, 3 - 3052031874 разів за 10 сек.
9 - 9990332244, 3 - 3330110748 разів за 10 сек.
11 - 10023831000, 3 - 3341277000 разів за 10 сек.
12 - 9953535195, 3 - 3317845065 разів за 10 сек.
15 - 10620179196, 3 - 3540059732 разів за 11 сек.
14 - 11508338505, 3 - 3836112835 разів за 11 сек.
6 - 10843813557, 3 - 3614604519 разів за 11 сек.
16 - 10585402395, 3 - 3528467465 разів за 11 сек.
17 - 10531870068, 3 - 3510623356 разів за 11 сек.
7 - 12815210439, 3 - 4271736813 разів за 12 сек.
5 - 13185095910, 3 - 4395031970 разів за 12 сек.
4 - 14134397661, 3 - 4711465887 разів за 12 сек.
10 - 14089338228, 3 - 4696446076 разів за 12 сек.
13 - 16604568117, 3 - 5534856039 разів за 13 сек.
Усі потоки завершили роботу. Починаємо новий цикл.
Введіть крок роботи потоків (0 для виходу)
Диспетчер задач
Файл Параметры Вид
Процессы Производительность Журнал приложений Автозагрузка Пользователи Подробности Службы
ЦП 4% 3,22 ГГц
Память 11,6/15,7 ГБ (74%)
Диск 0 (C:) SSD 0%
Диск 1 (D:) SSD 0%
Ethernet vEthernet (Default S. О: 0 П: 0 кбит/с
Ethernet Ethernet О: 0 П: 0 кбит/с
Графический п Intel(R) UHD Graphic 8%
ЦП 13th Gen Intel(R) Core(TM) i7-13620H
% использования более 60 секунд 100%
Использование 4%
Скорость 3,22 ГГц
Процессы 201
Потоки 3724
Дескрипторы 107861
Время работы 1:06:46:13
Базовая скорость: 2,40 Г...
Сокетов: 1
Ядра: 10
Логических процессоров: 16
Виртуализация: Вклю...
Кэш L1: 800 КБ
Кэш L2: 8,8 МБ
Кэш L3: 24,0 ...
Меньше
Открыть монитор ресурсов

Контрольні запитання.

1. Які програми називаються багатопоточними Багатопоточними називаються програми, які містять дві або більше незалежних послідовностей інструкцій, що виконуються одночасно в межах одного загального адресного простору процесу. Механізм дії полягає у розподілі процесорного часу між цими послідовностями за допомогою планувальника завдань операційної системи. Кожен потік отримує власний лічильник команд, стек викликів та набір регістрів для ізольованого виконання алгоритму. При цьому всі потоки мають спільний доступ до області купи (heap). Купа є загальним сегментом оперативної пам'яті, який операційна система виділяє для динамічного зберігання об'єктів під час роботи всієї програми, що робить ці об'єкти видимими для кожного створеного потоку . Спільний доступ до глобальних змінних батьківського процесу вимагає точної синхронізації пам'яті для уникнення станів гонитви. Стан гонитви (race condition) є апаратною проблемою, яка виникає при одночасному намаганні кількох потоків прочитати, модифікувати та записати дані в одну і ту саму комірку пам'яті. У такій ситуації кінцевий результат переписується останнім потоком і повністю залежить від непередбачуваного порядку виконання машинних інструкцій процесором .

2. Засоби організації багатопоточних програм у мові Java Базовими інструментами є клас Thread та інтерфейс Runnable. Для запуску обчислень створюється екземпляр класу Thread та викликається метод start(). Механізм дії ґрунтується на тому, що віртуальна машина Java (JVM) інтерпретує цей виклик. JVM працює як програмне середовище виконання, яке бере апаратно-незалежний байт-код Java і транслює його у машинні інструкції конкретного процесора під час роботи програми. Під час виклику методу start() JVM звертається до ядра операційної системи з вимогою виділити нативний системний потік. Нативний системний потік є реальною одиницею виконання на рівні ядра ОС (наприклад, потік Windows API або POSIX-потік у Linux), якій диспетчер завдань ОС безпосередньо виділяє час на фізичних ядрах

процесора. Для захисту спільних даних використовується модифікатор synchronized, який гарантує ексклюзивне захоплення блоку коду або методу лише одним потоком одночасно. Цей модифікатор реалізує алгоритм монітора на рівні заголовка об'єкта в пам'яті. Монітор працює як низькорівнева структура даних синхронізації, нерозривно пов'язана з конкретним об'єктом. Він містить лічильник входу та внутрішню чергу з ідентифікаторами потоків, які призупинені і очікують на звільнення об'єкта. Також застосовується ключове слово volatile, яке змушує процесор ігнорувати L1-кеш. L1-кеш є найшвидшою апаратною пам'яттю першого рівня, що фізично вбудована в мікросхему ядра процесора для тимчасового зберігання локальних копій змінних з метою прискорення доступу. Заборона кешування означає, що потік зобов'язаний зчитувати та записувати значення змінної виключно в головну оперативну пам'ять (RAM), що забезпечує миттєву видимість оновлених даних для всіх інших потоків в системі .

3. Засоби організації багатопоточних програм у мові C# Основними засобами є клас Thread з простору імен System.Threading та клас Task для високорівневого програмування. Програміст передає делегат у конструктор Thread і викликає метод Start(). У цей момент середовище виконання CLR (Common Language Runtime) взаємодіє з Windows API (наприклад, через системну функцію CreateThread), виділяючи ресурси операційної системи для паралельного виконання. Для синхронізації застосовується ключове слово lock, яке є синтаксичною обгорткою над системним класом Monitor і блокує виконання інших потоків до моменту звільнення критичної секції. Системний клас Monitor є вбудованим механізмом середовища .NET, який викликає системні переривання для блокування ділянки пам'яті об'єкта, суворо забороняючи іншим потокам виконувати інструкції між викликами своїх внутрішніх методів Enter та Exit. Модифікатор volatile в C# так само встановлює апаратний бар'єр пам'яті для атомарних операцій зчитування. Атомарні операції є неподільними процесорними командами, які виконуються за один такт синхронізації або виконуються повністю, або не виконуються

зовсім. Планувальник ОС апаратно не може перервати виконання потоку посередині атомарної операції.

4. Засоби організації багатопоточних програм у мові Ada

Багатопоточність реалізована за допомогою ключових слів task для обчислювальних процесів та protected для захисту структур даних. Механізм дії відрізняється фундаментально. Виконавче середовище Ada (Run-Time Environment) має власний планувальник завдань. Задачі взаємодіють між собою через точки входу entry за алгоритмом рандеву – синхронної передачі даних, коли один потік чекає на готовність іншого. Захищені об'єкти protected інкапсулюють змінні та дозволяють модифікувати їх лише через спеціальні процедури, гарантуючи взаємовиключний доступ на рівні компілятора без необхідності ручного створення м'ютексів. М'ютекс (від mutual exclusion) є базовим примітивом синхронізації операційної системи, який функціонує як бінарний прапорець зайнятості ресурсу. При захопленні м'ютекса одним потоком всі інші потоки, що звертаються до захищеного ресурсу, примусово переводяться ядром ОС у стан сну до моменту розблокування прапорця.

Різниця між вбудованою багатопоточністю (Ada) та бібліотечною (Java, C#)

У мовах C# та Java граматика та синтаксис є суто послідовними. Компілятори цих мов не розуміють поняття паралелізму. Всі багатопоточні операції реалізуються через зовнішні виклики до стандартних бібліотек платформи (CLR або JVM), які просто транслюють команди до операційної системи. Якщо програміст забуде поставити модифікатор volatile або блок lock, компілятор Java чи C# не видасть жодної помилки і успішно збере програму, яка згодом працюватиме некоректно через неконтрольований доступ до пам'яті.

У мові Ada багатопоточність є частиною мовного стандарту. Це означає, що концепції задач та їхньої синхронізації інтегровані безпосередньо в офіційну специфікацію ядра мови, так само як базові цикли чи умовні оператори, і розпізнаються компілятором без підключення зовнішніх

бібліотек. Компілятор GNAT розпізнає багатопоточність на рівні синтаксичного та семантичного аналізу. Синтаксичний аналіз дозволяє компілятору перевірити правильність написання ключових слів паралелізму згідно з жорсткою граматикою мови. Семантичний аналіз гарантує логічну узгодженість коду. Компілятор фізично аналізує структуру взаємодії потоків і заблокує збірку програми з видачею помилки, якщо виявить спробу прямого звернення до даних всередині захищеного об'єкта поза офіційними процедурами доступу. Виконавче середовище бере на себе всю відповідальність за створення черг очікування, розподіл пріоритетів та блокування, що робить код передбачуваним та незалежним від того, як працюють нативні потоки конкретної версії Windows чи Linux.